OPENING | ABOUT 3 MINUTES

Metso SCM demo

A presenter script for the application and agent already deployed.

Show the business request, a real incident, human-controlled recovery, the health workflow, the saved report and two contrasting code reviews. Allow roughly 25-35 minutes; live alert and agent processing can take longer.

Open these tabs

Material-request app / Azure SRE Agent

Material Request Health report

Good change: zhenbzha/sre-agent#1 / Bad change: zhenbzha/sre-agent#2

Use the existing onboardinglab-agent. Nothing needs to be redeployed, initialized or reconfigured. The only terminal actions are the live fault and reset on page 5.

1. Introduce the business story

> **SAY**
>
> A plant needs materials to keep its operations moving. This application records the request and gives the requester a reference number. Today I will show what happens when the application is still running but requests can no longer be saved, and how Azure SRE Agent helps us investigate.

2. Save one real request

1. In Material requests, briefly show the saved-request list, search and plant filter.
2. Click New request. Enter Screen panel, quantity 6, plant Nantes.
3. Click Submit request once. Point out Request saved and its generated request number.
4. Locate that same number in the list. Show the material, quantity and plant.

> **SAY**
>
> This is a real database-backed request, not a mock screen. We can see the saved record. Now let us introduce a controlled infrastructure problem.

> **Before proceeding**
>
> If the request does not save, do not inject the fault. Restore the healthy baseline first. All records are synthetic; this is not a production order-management or SAP integration demo.

SCENARIO 1 | INCIDENT INVESTIGATION

From failed request to cause

Let the real alert start the investigation. Do not give the agent the answer.

3. Introduce the failure

1. Start a ten-minute timer. Run Inject on page 5 and wait for completion.
2. In the app, open Demo controls > Edit values. Enter Wear liner, quantity 4, plant Lille, then choose Use these values.
3. Click Start simulation. Show the Failed counter increasing and the submission failure message.
4. Point out the healthy process indicator alongside the failed requests.

> **SAY**
>
> The plant cannot save its request, but the web process is still alive. A basic health endpoint alone would miss the business impact.

4. Open the automatically routed investigation

1. In SRE, open the new checkout-failures alert's investigation. It may be attached to an existing thread.
2. Check the alert time and messages: show this run, not an older investigation.
3. Show the agent gathering request telemetry, PostgreSQL dependency evidence and Azure resource configuration.

> **SAY**
>
> The alert started this investigation automatically. The agent is comparing application symptoms with infrastructure state, rather than guessing from an error message.

5. Show the evidence behind the diagnosis

Wait for the actual finding. Show the failed requests, database timeouts and the specific outbound NSG rule blocking PostgreSQL TCP 5432. A generic database-error summary is not the same result.

> **SAY**
>
> The useful result is not simply that the database is unreachable. It is the specific rule and network path that explain why these requests fail.

> **If attention is needed**
>
> Approve only the intended pending operation. To resume an interrupted thread: 'Continue investigating the current alert using fresh evidence. Do not change resources.' Never supply the diagnosis.

> **Keep the incident bounded**
>
> At ten minutes, stop simulation and reset even if investigation is incomplete. If you show an earlier successful thread instead, label it as the recorded rehearsal.

RECOVERY + SCENARIO 2 | HEALTH WORKFLOW

Recover, then check the service

You perform the recovery. The agent checks the operational evidence.

6. Restore and demonstrate the business operation

1. In Demo controls, click Stop simulation first. This avoids creating a batch of successful duplicate requests after recovery.
2. Run Reset on page 5 and wait for completion. Note the time.
3. Return to Material requests > New request. Submit Filter cartridge, quantity 3, plant Lyon.
4. Show the new Request saved number and its matching row in the list.

> **SAY**
>
> I am controlling the recovery. The new saved request shows that the business operation works again; a green process-health check alone would not prove that.

7. Ask the incident thread to confirm recovery

> The operator completed reset at [time and time zone]. Check request and PostgreSQL dependency telemetry after that time. Confirm whether the service has recovered, state any evidence gaps, and check this alert's current monitor condition. Do not change resources.

Show fresh successful requests and dependencies when they arrive. Alert resolution is delayed: allow roughly 10-15 minutes or longer after the last failure. Continue with the remaining scenarios while waiting; Resolved is different from Acknowledged or Closed.

8. Run the existing health workflow once

> **SAY**
>
> Incident investigation reacts to a failure. The health workflow gives us a repeatable review of the service without waiting for someone to start a troubleshooting chat.

1. In the SRE sidebar, open Automation.
2. Select the checkbox for reservation-daily-health-report, then click Run now. This is the existing SCM health task; its internal name is unchanged.
3. Click the task name to open its executions, then the newest Thread name link.
4. Show the actual findings: evidence window, request outcomes, PostgreSQL dependencies and any risks or missing data.

> **What to explain**
>
> The recent incident may still appear in the report's historical window. A Success badge only proves the task was handed to a thread: read the thread's findings. Leave the recurring task Off/Paused and its mode Review; do not turn on its schedule.

SCENARIOS 3 + 4 | REPORT AND CHANGE REVIEW

Make the evidence reusable

Finish with a saved operational view and the risk in the next code change.

9. Refresh the saved Live Report

> **SAY**
>
> This report gives us a repeatable view of request outcomes and database behavior.

1. Open Material Request Health and click Refresh.
2. Show the request summary, trend and outcome breakdown, then the PostgreSQL summary and trend. Point out the incident and recovery intervals.
3. Show baseline coverage: freshness, sparse history and evidence limitations.

Historical failures remain visible after recovery. Telemetry counts attempts, not just saved rows.

10. Compare the two sample PRs

Good: open zhenbzha/sre-agent#1. Show the database deadline changing from 5000 to 4500 milliseconds with matching documentation/tests. In SRE, show its completed review and PASS explanation.

Bad: open zhenbzha/sre-agent#2. Show await client.end() before forced cleanup. Show its SRE BLOCK verdict: hanging cleanup can hold a response and concurrency slot beyond the deadline.

> **SAY**
>
> These are the recorded reviews. Both changes are small, but the second can prolong an outage. The agent explains the operational risk from the actual code.

> **For a fresh review**
>
> Close and reopen each PR. Follow Actions > SRE Agent PR validation, then the new ticketing-pr-validation threads in SRE. Match PR numbers and head commits. Do not merge.

A green delivery job is not a PASS verdict. BLOCK is advisory, not an enforced merge gate.

11. Close the story

> **SAY**
>
> We have followed a request through failure and recovery, then used the service evidence for health checks, reporting and change review. The operator remains in control.

Finish with simulation stopped, reset complete, a new request saved and the health schedule Off/Paused. Check the alert; disclose if it is still resolving.

PRESENTER ACTION CARD | NOT PART OF THE TALK TRACK

Only two terminal actions

Use your existing signed-in PowerShell terminal. No setup block is required.

Both commands use the existing repository helper and explicitly target this demo. They change and restore the controlled fault; they do not redeploy the application or initialize the database.

Inject - use at step 3

```
& 'C:\Users\zhenzh\projects\metso\sre-agent\labs\onboardinglab\scripts\fault.ps1' `
  -Action inject `
  -Subscription 'db4948d5-b90c-47d4-91b5-d8c4c43493d2' `
  -ResourceGroup 'rg-flu-xzlxm7twvdxao' `
  -WorkloadOption 'app-service-postgresql' `
  -NetworkSecurityGroupName 'flu-xzlxm7twvdxao-app-nsg' `
  -NamePrefix 'flu-xzlxm7twvdxao'
```

Wait for 'Fault inject completed' before starting browser simulation. If the helper refuses because an earlier alert is still fired, do not bypass it; reset and allow that alert to resolve.

Reset - use at step 6, or whenever the demo is interrupted

```
& 'C:\Users\zhenzh\projects\metso\sre-agent\labs\onboardinglab\scripts\fault.ps1' `
  -Action reset `
  -Subscription 'db4948d5-b90c-47d4-91b5-d8c4c43493d2' `
  -ResourceGroup 'rg-flu-xzlxm7twvdxao' `
  -WorkloadOption 'app-service-postgresql' `
  -NetworkSecurityGroupName 'flu-xzlxm7twvdxao-app-nsg' `
  -NamePrefix 'flu-xzlxm7twvdxao'
```

> **Always finish here**
> Stop browser simulation before reset. Keep this reset command available, and run it if you abandon the incident or anything goes wrong. The browser's ten-minute traffic limit does not reset the fault. After reset, submit one request and show its saved row.

If recovery does not work

Do not claim the environment is healthy. Check that the reset helper completed; in Azure, confirm the existing PostgreSQL server is Ready and the controlled rule is Allow. A stopped database is a different problem. Keep the database private and do not change agent permissions to force a result.

Process-health endpoint / Dedicated Azure resource group

Microsoft Learn: Run now and execution threads

Presenter only: do not upload this script or its fault instructions into SRE knowledge.